

Hands-on Activity 11.1

Basic Algorithm Analysis

Course Code: CPE010

Program: Computer Engineering

Course Title: Data Structures and Algorithms

Date Performed: Oct 20, 2025

Section: CPE21S4

Date Submitted: Oct 20, 2025

Name(s): Punay, Heidee S.

Instructor: Engr. Jimlord Quejado

A. Output(s) and Observation(s)

ILO A: Measure the runtime of algorithms using theoretical analysis

```
#include <iostream>
#include <vector>
#include <chrono>
using namespace std;

int search(const vector<int>& x, int target) {
    for (int i = 0; i < x.size(); i++) {
        if (x[i] == target) return i;
    }
    return -1;
}

bool diff(const vector<int>& x, const vector<int>& y) {
    for (int i = 0; i < y.size(); i++) {
        if (search(x, y[i]) != -1) {
            return false;
        }
    }
    return true;
}

void test_diff(int size) {
    vector<int> x, y;
    for (int i = 0; i < size; i++) {
        x.push_back(i);
        y.push_back(i + size);
    }

    auto start = chrono::high_resolution_clock::now();
    bool result = diff(x, y);
    auto end = chrono::high_resolution_clock::now();

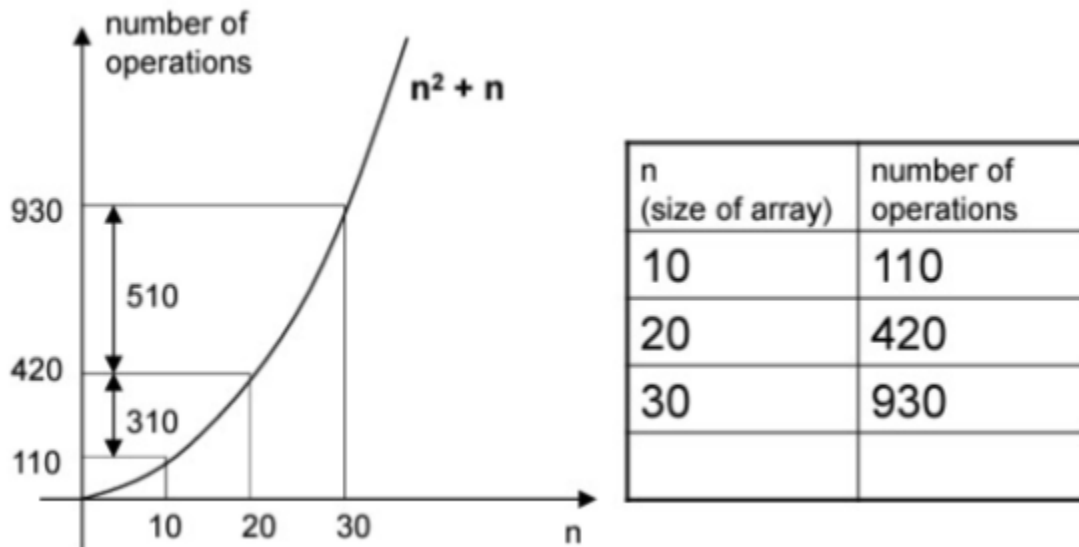
    cout << "Size: " << size << " | Result: " << (result ? "Different" : "Overlap")
         << " | Time: " << chrono::duration_cast<chrono::microseconds>(end -
start).count()
         << " microseconds" << endl;
}

int main() {
    test_diff(10);
    test_diff(20);
    test_diff(30);
    return 0;
}
```

Output:

```
Size: 10 | Result: Different | Time: 1 microseconds
Size: 20 | Result: Different | Time: 4 microseconds
Size: 30 | Result: Different | Time: 10 microseconds

Process finished with exit code 0
```



Analysis:

In this graph, the number of operation increases as the size of the array gets bigger this means that the program gets slower as the input gets bigger. The program checks every item in one list against every item in the other.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

```
#include <iostream>
#include<vector>
#include<chrono>
#include<random>
#include<algorithm>
#include<numeric>

class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;
public:
    // Constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v) {}

    // Getters
    auto get_name() { return name; }
    auto is_vaccinated() { return vaccinated; }

    // Two people are same if they have the same name
    bool operator==(const Student& p) const {
        return this->name == p.name;
    }
}
```

```

// The ordering of a set of people is defined by their name
bool operator<(const Student& p) const {
    return this->name < p.name;
}

bool operator>(const Student& p) const {
    return this->name > p.name;
}
};

auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;

    return Student(random_name, is_vaccinated);
}

bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();

    while (true) {
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);

        // Return true if the Student is found in the sequence and not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;
        else if (mid_element == P && mid_element.is_vaccinated() == true)
            return false;
        else if (mid_element > P)
            std::advance(last, -mid_element_index);
        if (mid_element < P)
            std::advance(first, mid_element_index);

        // Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1)
            return true;
    }
}

void search_test(int size, Student p) {
    std::vector<Student> people;

    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    std::cout << "Time taken to search = "

```

```

        << std::chrono::duration_cast<std::chrono::microseconds>(end -
begin).count()
        << " microseconds" << std::endl;

    if (search_result)
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
        << ") needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
        << ") does not need vaccination." << std::endl;
}

int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);

    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);

    return 0;
}

```

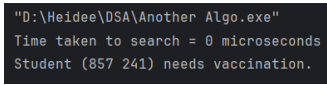
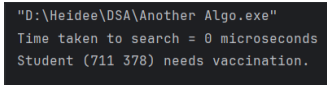
Output:

```

"D:\Heidee\DSA\Another Algo.exe"
Time taken to search = 1 microseconds
Student (858 99) needs vaccination.
Time taken to search = 4 microseconds
Student (858 99) needs vaccination.
Time taken to search = 6 microseconds
Student (858 99) needs vaccination.

Process finished with exit code 0

```

Input Size N	Execution Speed	Screenshot	Observation(s)
10	0 microseconds		With only 10 students on the list, it has finished the search so fast that it only has an execution speed of 0 microseconds. This results that the binary search is efficient for small inputs.
100	0 microseconds		The search is still the same as the first one, only that this has a size of 100, but it still runs smoothly and faster the same as the first

			one.
1000000	none	"D:\Heidee\DSA\Another Algo.exe"	In my observation, it didn't show anything, like how many seconds the execution speed is, but I think that it is because of how large the size of the student list is that it takes longer to search, and my device can't handle that much size.

This table shows the execution speed of the different input sizes and how it affects the execution speed. Like the third input, it didn't show any execution speed compared to the two that have only a size of 10 and 100. This proves that it is slow, or it shows issues when the size is so large.

B. Answers to Supplementary Activity

Problem 1:

```
#include <iostream>
#include <vector>
#include <chrono>

bool Unique(const std::vector<int>& A) {
    int n = A.size();
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] == A[j])
                return false;
        }
    }
    return true;
}

int main() {
    std::vector<int> A = {2, 4, 6, 8, 10};

    auto start = std::chrono::steady_clock::now();
    bool result = Unique(A);
    auto end = std::chrono::steady_clock::now();

    std::cout << "Time taken = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - start).count()
              << " microseconds" << std::endl;

    std::cout << "Array is " << (result ? "unique." : "not unique.") << std::endl;
    return 0;
}
```

Output:

```
"D:\Heidee\DSA\Supple HOA11.exe"
Time taken = 0 microseconds
Array is unique.
```

```
"D:\Heidee\DSA\Supple HOA11.exe"  
Time taken = 0 microseconds  
Array is not unique.
```

This program checks if all the numbers in a list are different from each other. It checks each number and compares it with other numbers. If it finds 2 numbers that are the same, it is not unique, and if it didn't find any numbers that are the same then the array is unique.

Problem 2:

```
#include <iostream>  
#include <chrono>  
  
int rpower(int x, int n) {  
    if (n == 0)  
        return 1;  
    else  
        return x * rpower(x, n - 1);  
}  
  
int brpower(int x, int n) {  
    if (n == 0)  
        return 1;  
    if (n % 2 == 1) {  
        int y = brpower(x, (n - 1) / 2);  
        return x * y * y;  
    } else {  
        int y = brpower(x, n / 2);  
        return y * y;  
    }  
}  
  
int main() {  
    int x = 2;  
    int n = 1000;  
  
    auto start1 = std::chrono::steady_clock::now();  
    int result1 = rpower(x, n);  
    auto end1 = std::chrono::steady_clock::now();  
  
    std::cout << "rpower result = " << result1 << std::endl;  
    std::cout << "Time taken by rpower = "  
        << std::chrono::duration_cast<std::chrono::microseconds>(end1 -  
start1).count()  
        << " microseconds" << std::endl;  
  
    auto start2 = std::chrono::steady_clock::now();  
    int result2 = brpower(x, n);  
    auto end2 = std::chrono::steady_clock::now();  
  
    std::cout << "brpower result = " << result2 << std::endl;  
    std::cout << "Time taken by brpower = "  
        << std::chrono::duration_cast<std::chrono::microseconds>(end2 -  
start2).count()  
        << " microseconds" << std::endl;  
  
    return 0;  
}
```

Output:

```
"D:\Heidee\DSA\SuppleP2 H0A11.exe"
rpower result = 0
Time taken by rpower = 61 microseconds
brpower result = 0
Time taken by brpower = 0 microseconds
```

The rpower multiplies the number by itself repeatedly until it reaches the power. The brpower, it divides and conquers, it breaks the problem into smaller parts and reuses the results. Both have the same results but with different time taken. The brpower is much faster than the rpower.

C. Conclusion & Lessons Learned

To conclude, in this activity, we have learned how to check the execution speed or the time taken to search for different sizes of the array. This also helps determine the device used to check how much size or data it can handle, or run the program. And I also learned the difference between rpower and brpower and how much they differ in how they function and also the time difference.

D. Assessment Rubric

E. External References